

Plan de Dirección y Desarrollo de Software: SIGCAT

(Sistema de Registro Gráfico de Predios y Detección Automática de Solapamiento Catastral)

Institución: Universidad Antonio José de Sucre (UNIAJS) - Programa de Ingeniería de Sistemas. **Asignatura:** Ingeniería de Software. **Docente:** Ing. Juan Manuel Palacio.

1. Asignación de Roles del Equipo

Para garantizar la entrega del prototipo funcional y distribuir equitativamente el trabajo entre los 5 miembros, se definen las siguientes responsabilidades estructuradas bajo un enfoque de desarrollo ágil:

Cristian Morales – Técnico y Arquitecto:

Responsable de definir la arquitectura MVC, orquestar la integración entre los módulos de interfaz y la base de datos, configurar Maven (pom.xml) y generar el empaquetado final del sistema (.jar).

Andrés Díaz – Desarrollador Core y Lógica Espacial:

Encargado de la investigación e implementación de las herramientas matemáticas y geométricas, específicamente el algoritmo de Shoelace para el área y el motor de JTS Topology Suite para la validación de solapamientos.

Leider Barreto – Desarrollador de Interfaz Gráfica (UI/UX):

Responsable de la construcción de las vistas en JavaFX (archivos FXML), la interacción del Canvas para el dibujo de los polígonos mediante eventos de ratón y el feedback visual de errores al usuario.

Aldo Ibañez – Desarrollador de Persistencia (Base de Datos):

Encargado de diseñar el esquema relacional en SQLite, construir el patrón DAO (Data Access Object) y garantizar que las coordenadas y estados de los predios se guarden y recuperen correctamente.

José Sánchez – Analista de Calidad (QA) y Testing:

Responsable de diseñar los casos de prueba, ejecutar pruebas unitarias (JUnit) sobre los cálculos de áreas e intersecciones, y documentar los resultados del flujo de aprobación.

2. Alcance del Proyecto

El sistema es un prototipo funcional de escritorio enfocado en resolver el problema del registro catastral centralizado y los conflictos de linderos.

- **Incluye:**
 - Autenticación de usuarios por roles (Propietario y Funcionario).
 - Módulo de captura gráfica (Canvas) para el registro del perímetro de los predios.
 - Cálculo automático de área (Algoritmo Shoelace).
 - Detección preventiva de conflictos/solapamientos entre polígonos usando JTS Topology Suite.
 - Flujo de trabajo donde un funcionario catastral aprueba o rechaza solicitudes pendientes.
- **No incluye:**
 - Integración con mapas satelitales reales (se trabajarán coordenadas (x, y) relativas en el lienzo).
 - Despliegue en red local o nube (el prototipo utilizará una base de datos local embebida).
 - Registro de polígonos complejos con huecos internos o curvas.

3. Arquitectura y Modelo de Datos

El proyecto implementará el patrón arquitectónico **MVC (Model-View-Controller)** para separar la lógica geométrica de la interfaz visual, facilitando el trabajo en paralelo de los desarrolladores.

Esquema de Base de Datos Relacional (SQLite)

Se descarta el uso de texto plano para guardar coordenadas. Se implementarán las siguientes tablas:

1. **Usuarios:** id_usuario (PK), documento, nombre, password, rol (PROPIETARIO, FUNCIONARIO).
2. **Predios:** id_predio (PK), id_propietario (FK), id_funcionario (FK), area_calculada (DOUBLE), estado (PENDIENTE, APROBADO, RECHAZADO), fecha_registro.
3. **Vertices:** id_vertice (PK), id_predio (FK), orden (INT), coord_x (DOUBLE), coord_y (DOUBLE).

4. Fases de Implementación y Cronograma Detallado

El trabajo se estructurará en 4 Sprints (Fases) de dos semanas cada uno, con entregables específicos.

Fase 1: Fundamentos y Arquitectura Base

Objetivo: Configurar el entorno, conectar la base de datos y construir el acceso de usuarios.

- **Cristian:** Configura el proyecto en IntelliJ IDEA con Maven. Establece la estructura de paquetes (ui, logic, data).
- **Aldo:** Crea el script de inicio de SQLite (CREATE TABLE IF NOT EXISTS) y programa las clases DAO para conexión a base de datos mediante JDBC.
- **Leider:** Diseña la pantalla de Inicio de Sesión (Login.fxml) y rutea el menú principal según el rol.

Entregable: Repositorio en GitHub funcional; sistema compila, crea la base de datos y permite autenticarse.

Fase 2: Módulo Gráfico y Geometría Básica

Objetivo: Permitir el dibujo de predios y el cálculo matemático de superficies.

- **Leider:** Programa el Canvas en JavaFX. Captura los clics del propietario para dibujar puntos y líneas. Cierra el polígono al hacer clic en el punto inicial.
- **Andrés:** Recibe las coordenadas capturadas por Leider y codifica la clase ShoelaceCalculator.java para aplicar la fórmula de Gauss y devolver el área.
- **José:** Escribe pruebas unitarias pasándole coordenadas manuales (ej: un cuadrado perfecto) al método de Andrés para confirmar que el área sea matemáticamente exacta.

Entregable: Interfaz que permite dibujar un polígono y muestra su área en pantalla.

Fase 3: Lógica Espacial y Persistencia de Vértices

Objetivo: Implementar el corazón del sistema: la detección de solapamientos.

- **Leider & Cristian:** Integran la librería JTS Topology Suite. Convierten la lista de vértices de JavaFX en un objeto Polygon de JTS. Implementan el ciclo que verifica `poligonoNuevo.intersects(poligonoAprobado)`.
- **Aldo:** Implementa la inserción transaccional: al guardar un predio, se inserta un registro en la tabla Predios y múltiples registros en la tabla Vertices respetando su orden.
- **José:** Ejecuta pruebas de estrés geométrico (polígonos que se tocan en un vértice, polígonos superpuestos totalmente).
- **Entregable:** El sistema bloquea al propietario si intenta registrar un predio que invade territorio ya aprobado.

Fase 4: Flujo de Aprobación, Estabilización y Entrega

Objetivo: Completar el rol del funcionario y empaquetar el producto final.

- **Andrés:** Desarrolla el panel del Funcionario Catastral: una tabla que lista las solicitudes PENDIENTES, muestra su dibujo y permite cambiar el estado a APROBADO o RECHAZADO.
- **Cristian:** Configura el *Maven Shade Plugin* para exportar el proyecto completo en un único archivo .jar ejecutable (Fat JAR).
- **José & Todo el equipo:** Realizan pruebas de flujo End-to-End, redactan el manual de usuario, y corrigen los últimos bugs de la interfaz.
- **Entregable:** Archivo JAR ejecutable, manual de usuario y código fuente listo para sustentación.

5. Gestión de Riesgos Técnicos

Riesgo Detectado	Nivel	Plan de Mitigación (A cargo de)
Bloqueo del hilo de UI (JavaFX) al validar solapamientos contra muchos predios.	Alto	Cristian: Envolver la consulta a JTS en un bloque asíncrono (Task y Platform.runLater) para que la pantalla no se congele.
Curva de aprendizaje de JTS Topology Suite.	Medio	Leider: Iniciar el estudio de la API de JTS desde la Fase 1, sin esperar a la Fase 3, realizando scripts de prueba separados.
Pérdida de precisión o deformación del polígono si el usuario cambia el tamaño de la ventana.	Medio	Andrés: Bloquear el redimensionamiento de la ventana de dibujo (fijar el Canvas a 1024x768 píxeles).
Riesgo Detectado	Nivel	Plan de Mitigación (A cargo de)
Bloqueo del hilo de UI (JavaFX) al validar solapamientos contra muchos predios.	Alto	Cristian: Envolver la consulta a JTS en un bloque asíncrono (Task y Platform.runLater) para que la pantalla no se congele.
Curva de aprendizaje de JTS Topology Suite.	Medio	Leider: Iniciar el estudio de la API de JTS desde la Fase 1, sin esperar a la Fase 3, realizando scripts de prueba separados.
Pérdida de precisión o deformación del polígono si el usuario cambia el tamaño de la ventana.	Medio	Andrés: Bloquear el redimensionamiento de la ventana de dibujo (fijar el Canvas a 1024x768 píxeles).

6. Criterios de Terminación (Definition of Done)

Una tarea o fase solo se considerará terminada cuando cumpla estrictamente con lo siguiente:

1. El código compila sin errores en Java 17 LTS.
2. La funcionalidad no rompe ninguna sección previamente operativa.
3. El código ha sido subido a la rama principal (main) del repositorio.
4. No existen credenciales o rutas de archivos quemadas en el código (hardcoded) que impidan correr el sistema en la computadora del profesor.